

Hermes Virtual Software Company

Calculator Proof of Concept — End-to-End Implementation Report

Prepared by: Mohammad Ali Shkeir | Date: September 30 - October 1, 2026

1. Executive Summary

This report documents the end-to-end setup and first live test of a Hermes Agent-driven “virtual software company”: a multi-agent pipeline (Marketing → Sales → Business Analyst → Accounting → Project Manager → Developer → QA → DevOps) built to take a customer request from first contact through to a deployed, working application, with minimal human intervention beyond supervision and approvals.

The experiment used a simple calculator web application as the test case, per the agreed proof-of-concept scope. The result: the full pipeline ran end to end on real infrastructure, producing a real GitHub commit and a real, working, publicly reachable calculator application. Several genuine gaps and failure modes were also surfaced along the way — these are documented as findings, not hidden, since surfacing them is a core purpose of this exercise.

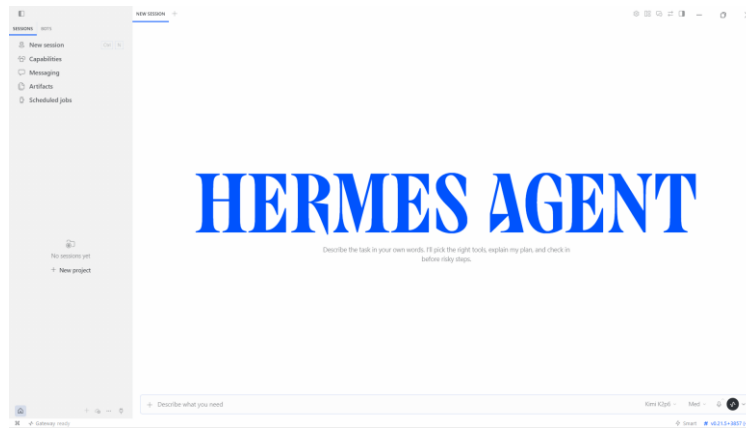
2. Architecture Decision: Hermes's Role

Before implementation began, three independent pieces of research (the baseline TabbeQ AI BRD/SRS, and two teammates' separate studies) reached different conclusions about what role Hermes Agent should play. This was resolved as ADR-0001:

- Phase 1 (this experiment): Hermes Agent is used as-is, directly, as the execution engine for the virtual company pipeline.
- Future / Phase 2+: if an in-app AI copilot is added to a production system later, it should be a small custom-built service inspired by Hermes's architecture — not the Hermes Agent product embedded directly — to keep production attack surface and scope minimal.
- Non-negotiable in all cases: no AI layer directly mutates production state without a human-reviewable trail; every action is logged; sensitive operations require approval.

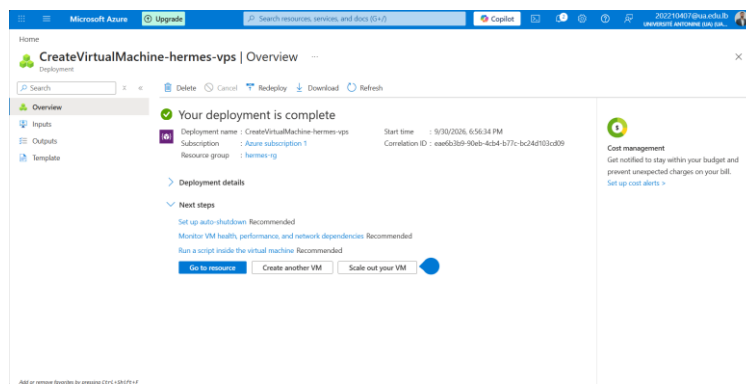
3. Phase 1 — Infrastructure

Hermes Agent was first installed locally on Windows to validate the tool itself. This surfaced and resolved two real Windows-specific bugs: a git long-path limit during repository clone, and an upstream bug in the `uv` Python package manager (astral-sh/uv #19622) where a managed-Python “minor version link” junction silently failed to resolve, blocking Python bootstrap. Both were diagnosed and fixed.



Hermes Agent Desktop, running locally on Windows after initial install.

The real target, however, was cloud hosting — the project's goal from the outset was an always-online Hermes instance, not one tied to a laptop. A VM was provisioned on Microsoft Azure (via the GitHub Student Developer Pack / Azure for Students credit, avoiding any credit-card requirement): Ubuntu Server 24.04 LTS, Standard B2ls_v2 (2 vCPU / 4 GiB RAM), region West Europe (chosen after closer regions rejected that SKU under the student subscription).



Azure deployment complete — the hermes-vps virtual machine.

Hermes Agent was then installed on the VM via its official Linux installer. A missing system library (libatomic1, required by the bundled Node.js binary) caused the first install attempt to fail; installing it and re-running the installer resolved this cleanly — the Linux install path was otherwise far smoother than the Windows one.

3.1 Making Hermes Persistent

Two systemd services were configured so Hermes survives SSH disconnects and VM reboots: `hermes-gateway` (messaging/cron, via Hermes's own `hermes gateway install`) and `hermes-serve` (the API/dashboard backend on port 9119, configured manually with dashboard login credentials for security).



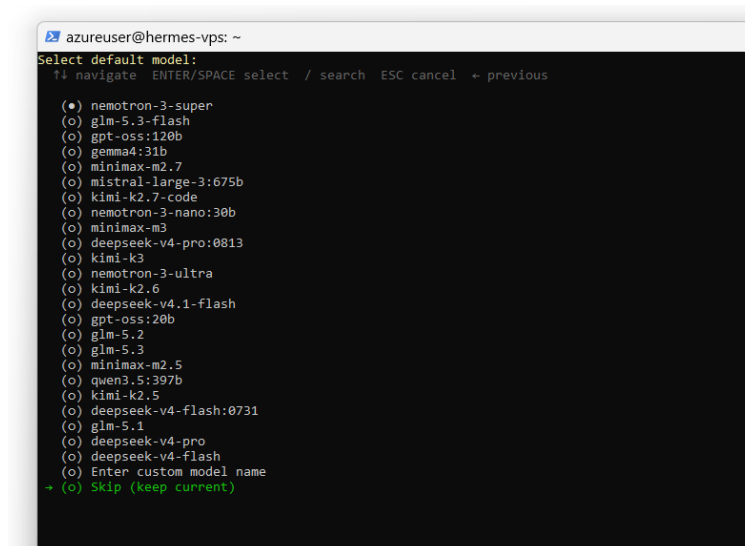
Hermes Agent running on the Azure VM via SSH, confirming a clean Linux install.

3.2 Remote Access Without SSH Dependency

The university/home network used blocked outbound SSH (port 22), so an alternative access path was built: Hermes Desktop's "Remote gateway" (plain HTTP) connection mode, authenticated with a dashboard username/password, reachable on port 9119 without needing any special network access. This became the normal way to use Hermes day to day; SSH is now only needed for occasional VM maintenance.

4. Phase 2 — Model Provider and GitHub

The model provider went through two iterations. Google Gemini (via Google AI Studio) was configured first, but its free tier hit a hard quota wall (HTTP 429 RESOURCE_EXHAUSTED) partway through the first live test — notably, a personal "Gemini Pro" subscription does not raise API quota, since that requires separate Google Cloud billing on the API project specifically. The provider was switched to Ollama Cloud (model: nemotron-3-super), which resolved the issue and was used for the rest of the experiment.



Selecting the Ollama Cloud default model after the Gemini quota wall.

GitHub access was configured via a fine-grained Personal Access Token, authenticated to the `gh` CLI on the VM. Two credential mistakes occurred and were corrected during setup: the token was twice pasted in a way that exposed it, requiring revocation and regeneration (standard practice after any credential exposure); and a `grep` command used to extract the token from the `.env` file was initially unanchored,

accidentally matching a commented-out placeholder line as well and corrupting the extracted value — fixed by anchoring the grep pattern.

5. Phase 3 — Multi-Agent Architecture

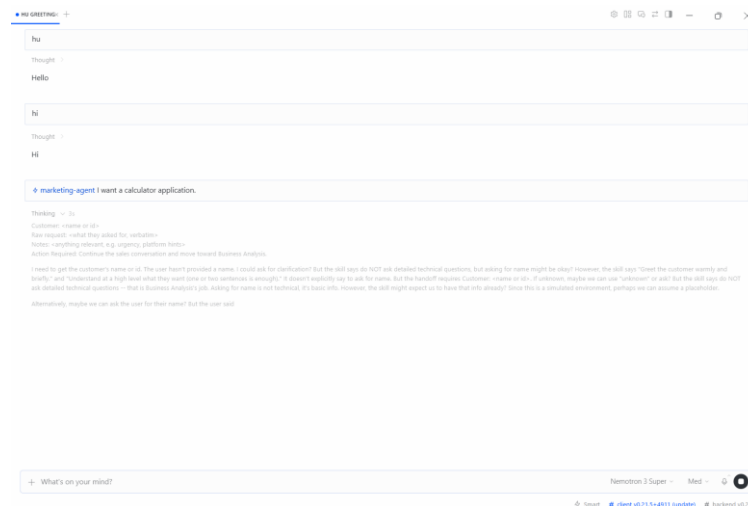
Eight Hermes “skills” (on-demand instruction documents, each a SKILL.md file) were written, one per department, matching the agreed scoped-down version of the original workflow design:

- marketing-agent — first contact, qualifies the request
- sales-agent — explains the process, confirms intent
- business-analyst-agent — asks clarifying questions, writes the PRD
- accounting-agent — simulates commercial/payment approval
- project-manager-agent — owns execution, delegates to the next three and tracks shared state
- developer-agent — implements the code and commits it
- qa-agent — tests against the PRD's acceptance criteria
- devops-agent — deploys the application and returns a live URL

Each skill hands off to the next via Hermes's `delegate\_task` tool (spawning a real subagent) with a structured context block, and a shared `PROJECT\_STATE.md` file was defined to track project status across agents.

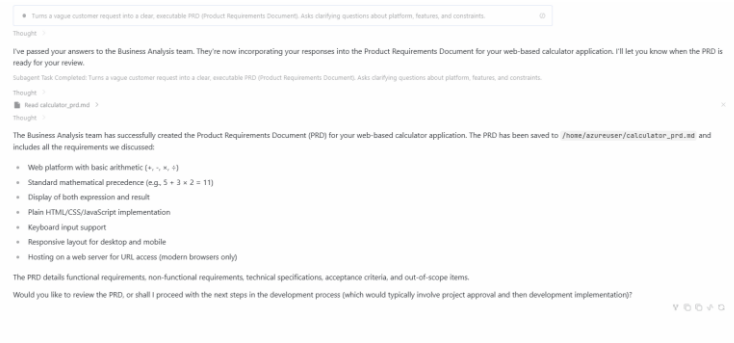
6. Phase 4 — Live Test: “I want a calculator application”

The pipeline was triggered by acting as the customer and invoking `/marketing-agent I want a calculator application.` The full chain ran through all eight stages.



Marketing agent's first response — greeting, scope check, and a request for the customer's name.

Sales asked five qualifying questions (platform, special functions, UI preferences, use case, timeline); Business Analysis followed with six more detailed clarifying questions (operator precedence, display format, tech stack, keyboard support, responsiveness, deployment target) and then produced a full PRD.



The Business Analyst's PRD summary, presented for customer approval.

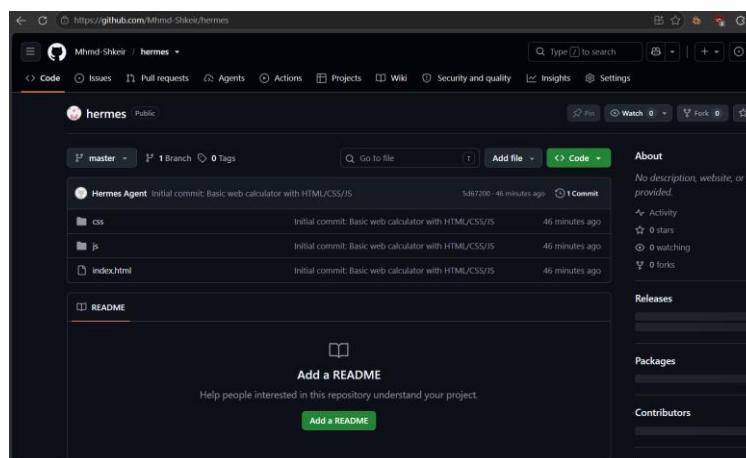
After approval, Accounting recorded simulated commercial approval, and the Project Manager took over: breaking the PRD into roughly 15 implementation tasks and delegating in sequence to Developer, QA, and DevOps.

7. Result

The Developer agent implemented the calculator in plain HTML/CSS/JavaScript (operator precedence, dual expression/result display, keyboard support, responsive layout, error handling for invalid input and division by zero). After an initial deployment failure (see Findings), the application was made persistent via a dedicated systemd service and the corresponding Azure firewall port, and the code was pushed to the customer's existing GitHub repository.



The deployed calculator, live at <http://40.113.162.135:8000>, correctly computing $7 + 6 = 13$.



The real commit (5d67200) pushed by Hermes Agent to github.com/Mhmd-Shkeir/hermes.

8. Findings and Lessons Learned

Several genuine gaps surfaced during the run — documented here rather than smoothed over, since this is the point of the exercise:

- Lossy handoffs: a parent agent sometimes summarized a subagent's reply instead of relaying it verbatim to the customer, hiding real content (e.g. Sales's actual questions) behind a collapsed UI element. Partially worked around live; the permanent fix is an explicit “relay verbatim” instruction added to the skill files.
- Claimed-but-not-done: at one point Marketing stated a handoff to Business Analysis had happened when it had not yet actually called `delegate_task`. When directly challenged (“did you actually hand off, or just say you would?”), the agent honestly identified the gap in its own reasoning trace and then performed the real handoff — a positive trust signal despite the initial error.
- Approval-gate friction: DevOps could not create a `systemd` service itself, because ``sudo systemctl`` matches Hermes's built-in dangerous-command list, and an unattended background subagent has no human available to approve it (by design, this fails closed/denied). The fix was applied by a human directly over SSH.
- GitHub repo creation via a fine-grained PAT returned HTTP 403, because such tokens generally cannot create new repositories unless scoped to “All repositories” with Administration permission. Worked around by using an existing repository instead of a newly created one.
- No agent had been explicitly assigned repository creation in the original skill design — a real gap identified only by running the workflow, not by reviewing the skill files in advance.

9. Next Steps

- Apply the verbatim-relay fix to all eight skill files before the next run.
- Run the doc's Phase 9 change-request test (e.g. “add a percentage button, change the color”) to validate the update/redeploy cycle, not just the initial build.
- Decide and document an explicit repository-creation owner among the eight agents.
- Grant the manager/stakeholder direct access: Hermes itself via the remote gateway (<http://40.113.162.135:9119>) and the live calculator (<http://40.113.162.135:8000>).
- If this pipeline is extended toward the larger TabbeQ AI ERP project later, revisit ADR-0001 before scoping any in-app AI features — per that decision, those should be custom-built, not embedded Hermes.